

TRAFFIC FLOW SIMULATION SYSTEM

Final Project Report

Submitted By:

Name: Anoshy fatima

Roll Number: FA-24/BS SE/200

Course: Software Construction and Development

Semester: 4th

TABLE OF CONTENT;

TRAFFIC FLOW SIMULATION SYSTEM	1
Final Project Report	1
Submitted By:	1
1. Introduction.....	2
2. Project Objectives	2
3. System Overview	2
Main Components	2
Vehicle Module	2
Traffic Light Module	2
Road and Intersection Module.....	2
Simulation Engine	3
Analytics Module	3
User Interface	3
4. Process Model Used.....	3
Justification of Agile Model.....	3
Development Sprints	3
Sprint 1	3
Sprint 2	3
Sprint 3	3
Sprint 4	4
5. Software Process Improvement (SPI)	4
Initial Problems	4
Improvements Made.....	4
Results of SPI	4
6. Version Control Implementation	4
Branch Structure.....	4
Advantages of Version Control.....	5
7. Lehman's Law Justification	5
8. Software Deployment	5
Deployment Components	6
Frontend.....	6

Backend	6
Database.....	6
Stored Data	6
Deployment Process	6
9. Refactoring and Legacy Code Removal	6
Before Refactoring	6
After Refactoring.....	7
Refactoring Benefits.....	7
10. Unit Testing	7
Unit Tests Performed	7
Vehicle Tests	7
Traffic Light Tests	7
Analytics Tests	7
Simulation Engine Tests	7
11. Automated Testing.....	8
Automated Tests Included.....	8
12. Exception Handling	8
Exceptions Handled.....	8
13. Peer Reviews and Inspections.....	8
Review Activities	8
Improvements After Review	9
14. Individual Contribution.....	9
Responsibilities Performed	9
Planning and Design.....	9
Backend Development.....	9
Frontend Development	9
Database Management.....	9
Testing and Debugging.....	10
Documentation.....	10
15. Learning Outcomes	10
Skills Learned.....	10
16. Conclusion	10

1. Introduction

Traffic congestion is one of the major problems in urban areas. Managing vehicle movement efficiently requires intelligent traffic monitoring and simulation systems. The Traffic Flow Simulation System was developed to simulate real-time traffic movement across roads and intersections controlled by traffic signals.

The system models vehicles, roads, intersections, and traffic lights while monitoring congestion levels and generating traffic analytics. The project was developed to demonstrate practical implementation of software engineering and software construction concepts including process models, testing, refactoring, deployment, version control, and software evolution.

2. Project Objectives

The major objectives of the project are:

- Simulate vehicle movement on roads
- Control traffic using traffic light systems
- Detect traffic congestion
- Generate traffic analytics
- Apply software engineering concepts practically
- Implement modular and maintainable architecture
- Demonstrate testing and deployment practices

3. System Overview

The Traffic Flow Simulation System consists of multiple software modules working together to simulate real-time traffic conditions.

Main Components

Vehicle Module

Handles vehicle creation, movement, speed, and positioning.

Traffic Light Module

Controls traffic signal timing and state transitions.

Road and Intersection Module

Represents road structures and traffic intersections.

Simulation Engine

Manages the simulation loop and system updates.

Analytics Module

Calculates congestion statistics and generates reports.

User Interface

Displays traffic simulation and allows user interaction.

4. **Process Model Used**

The Agile Software Development Model was used for project development.

Justification of Agile Model

Agile was selected because the project requirements evolved continuously during development. The project was divided into small modules and developed incrementally in multiple stages.

The Agile model supported:

- Continuous improvement
- Frequent testing
- Easier debugging
- Flexible feature addition
- Faster development cycles

Development Sprints

Sprint 1

- Project planning
- Basic simulation engine
- Vehicle movement implementation

Sprint 2

- Traffic light control system
- Road and intersection modules

Sprint 3

- Analytics and congestion detection
- Frontend visualization dashboard

Sprint 4

- Testing
- Refactoring
- Exception handling
- Deployment setup

The Agile process improved development efficiency and software quality.

5. Software Process Improvement (SPI)

Software Process Improvement was continuously applied during the project lifecycle.

Initial Problems

Initially, the system contained:

- Large monolithic functions
- Duplicate calculations
- Poor separation of modules

Improvements Made

The system was improved by:

- Separating simulation logic into modules
- Reducing duplicate congestion calculations
- Improving traffic light update efficiency
- Organizing controllers independently

Results of SPI

The improvements resulted in:

- Better maintainability
- Improved scalability
- Faster execution
- Cleaner architecture
- Easier debugging

6. Version Control Implementation

Git and GitHub were used for version control and source code management.

Branch Structure

The following branches were maintained:

- main
- development
- feature/simulation-engine
- feature/traffic-lights
- feature/analytics
- feature/ui

Advantages of Version Control

- Tracking code changes
- Maintaining code history
- Safe feature development
- Easier debugging and rollback
- Better project organization

Regular commits were made throughout development to maintain project history.

7. Lehman's Law Justification

The project demonstrates Lehman's Law of Software Evolution.

Lehman's Law states that software systems continuously evolve and become more complex over time unless maintained properly.

Initially, the system only simulated basic vehicle movement. As the project evolved, additional features were added including:

- Traffic signal management
- Congestion monitoring
- Analytics generation
- Real-time visualization
- Simulation controls

As the number of roads, intersections, and vehicles increased, system complexity also increased. This required continuous maintenance, optimization, and refactoring to maintain performance and scalability.

Therefore, the Traffic Flow Simulation System clearly demonstrates software evolution and increasing complexity over time.

8. Software Deployment

The system was deployed using a client-server architecture.

Deployment Components

Frontend

- HTML
- CSS
- JavaScript Dashboard

Backend

- FastAPI / Flask backend server

- Simulation engine processing

Database

- MySQL database

Stored Data

The database stores:

- Vehicle information
- Road data
- Traffic light states
- Simulation logs
- Analytics reports

Deployment Process

1. Backend server initialization
2. Database connection establishment
3. Frontend connection through REST APIs
4. Simulation execution through browser dashboard

The deployment setup ensured smooth communication between all system components.

9. Refactoring and Legacy Code Removal

Refactoring was performed to improve code quality and maintainability.

Before Refactoring

Initially, all simulation logic was implemented in a single large file which caused:

- Complex debugging
- Poor readability
- Tight coupling
- Duplicate code

After Refactoring

The project was reorganized into modular components:

- simulation/
- controllers/
- analytics/
- models/
- database/
- frontend/

Refactoring Benefits

- Improved readability
- Reduced complexity
- Easier maintenance
- Better scalability
- Improved modularity

Legacy and redundant code sections were removed during refactoring.

10. Unit Testing

Unit testing was implemented to validate important system functionalities.

Unit Tests Performed

Vehicle Tests

- Vehicle movement validation
- Vehicle stop functionality
- Position update testing

Traffic Light Tests

- Signal state transition testing
- Timer validation

Analytics Tests

- Congestion calculation testing
- Waiting time calculation

Simulation Engine Tests

- Start simulation test
- Pause simulation test
- Reset simulation test

Pytest framework was used for unit testing.

11. Automated Testing

Automated testing was used to improve reliability and reduce manual testing effort.

Automated Tests Included

- API endpoint testing
- Database connection testing
- Traffic signal updates

- Vehicle generation testing
- Simulation initialization testing

Automated testing ensured that new changes did not break existing functionality.

12. Exception Handling

Exception handling mechanisms were implemented to improve software reliability and stability.

Exceptions Handled

- Invalid vehicle routes
- Missing intersections
- Database connection failures
- Traffic signal timing errors
- Invalid simulation states
- Overflow of vehicle objects

Validation checks and try-catch mechanisms prevented application crashes and improved user experience.

13. Peer Reviews and Inspections

Even though the project was completed individually, self-review and inspection techniques were applied throughout development.

Review Activities

- Code walkthroughs
- Logic inspection
- UI inspection
- Testing review
- Bug analysis

Improvements After Review

- Improved variable naming
- Removed duplicate code
- Optimized simulation update loops
- Improved UI readability
- Reduced unnecessary calculations

Regular inspection improved software quality and maintainability.

14. Individual Contribution

This project was completed individually without any team members. All phases of the project were handled independently.

Responsibilities Performed

Planning and Design

- Requirement analysis
- UML diagram creation
- Architecture planning

Backend Development

- Simulation engine
- Vehicle logic
- Traffic light control
- Congestion algorithms

Frontend Development

- Dashboard design
- Traffic visualization
- User controls

Database Management

- Schema design
- Data storage
- Analytics management

Testing and Debugging

- Unit testing
- Automated testing
- Exception handling
- Bug fixing

Documentation

- Final report preparation
- Diagram generation
- Deployment documentation

15. Learning Outcomes

The project provided practical exposure to software engineering and software development concepts.

Skills Learned

- Agile software development
- Git and version control
- UML and ER diagram design
- Modular software architecture
- Refactoring techniques
- Unit and automated testing
- Deployment management
- Exception handling
- Simulation system development
- Problem-solving and debugging

The project also improved understanding of scalable system design and software evolution.

16. Conclusion

The Traffic Flow Simulation System successfully demonstrated the implementation of software construction and development principles. The project combined simulation logic, modular architecture, testing, deployment, and software engineering methodologies into a complete software system.

The system achieved its objectives by simulating real-time traffic behavior, managing congestion, and generating analytics while following professional software engineering practices. The project also provided valuable practical experience in software design, testing, deployment, and maintenance.